

LAB 2 – INTELLIGENT DECISION-MAKING IN ROBOTICS

IGNACIO GONZÁLEZ MELÉNDEZ

NIA: 100522976

Exercise 1:

In this exercise when we execute the code, we get this output.

```
• (venv) ignagonmell@DESKTOP-SVHOVNC:~$ python EX1.py
The prisoner's dilemma
[(array([0., 1.]), array([0., 1.])))
```

This means that the Nash equilibrium for both prisoners is a betrayal strategy, because if one of them cooperates the best option for the other one is to betray. So, the only way to get the equilibrium is by a betrayal dominant strategy.

Exercise 2:

In this exercise we have player 1, who prefers theater, and player 2, who prefers ballet.

The payoff matrices are the following:

P1	[[4, 0],	P2	[[2, 0],
	[0, 2]]		[0, 4]]

The possible strategies for an equilibrium are:

- Both go to theatre, which would give 4 points for Player 1 and 2 points for Player 2. (T, T)
- Then we give a pure ballet strategy, this would give 2 points to Player 1 and 4 points to Player 2. (B, B)
- Lastly, we have a mixed strategy. P1 plays T with probability $\frac{2}{3}$ and B with $\frac{1}{3}$, P2 plays T with probability $\frac{1}{3}$, B with $\frac{2}{3}$.

This can be calculated in the following way. Let us say that P2 is indifferent so P1 plays T with a probability p , this means that $2p=4(1-p)$, so $p = \frac{2}{3}$. Also, if P1 is indifferent P2 plays T with a probability “ q ”, this means that $4q = 2(1-q)$, so $q = \frac{1}{3}$.

For this case (mixed strategy), I needed to use AI as I did not understand at first why the output had three possible equilibriums.

This can be seen in the output terminal as

```
(venv) ignagonmell@DESKTOP-SVHOVNC:~$ python EX2.py
Shows problem
[(array([1., 0.]), array([1., 0.])), (array([0., 1.]), array([0., 1.])), (array([0.66666667, 0.33333333]), array([0.33333333, 0.66666667]))]
```

Exercise 3:

The output is.

```
(venv) ignagonmell@DESKTOP-SVHOVNC:~$ python EX3.py
The prisoner's dilemma
Utility for P1 = -2.0
Utility for P2 = -0.5
```

Calculating the utilities by hand

$$U1 = \sigma P1 * A * \sigma P2^T = [1, 0] * \begin{bmatrix} -1 & -3 \\ 0 & -2 \end{bmatrix} * [0.5, 0.5]^T = -2$$

$$U2 = \sigma P1 * B * \sigma P2^T = [1, 0] * \begin{bmatrix} -1 & 0 \\ -3 & -2 \end{bmatrix} * [0.5, 0.5]^T = -0.5$$

Exercise 4:

If we are using Nash equilibrium: $\sigma P1 = [0, 1]$ and $\sigma P2 = [0, 1]$.

$$U1 = \sigma P1 * A * \sigma P2^T = [0, 1] * \begin{bmatrix} -1 & -3 \\ 0 & -2 \end{bmatrix} * [0, 1]^T = -2$$

$$U2 = \sigma P1 * B * \sigma P2^T = [0, 1] * \begin{bmatrix} -1 & 0 \\ -3 & -2 \end{bmatrix} * [0, 1]^T = -2$$

For this case, the equilibrium payoff would be (-2, -2). It is different from the one in exercise 3 because in this case both prisoners are looking for their best payoff, in which both get the best possible result (Nash equilibrium).

Exercise 5:

In this code, we have a long output because the code is trying several times to each an equilibrium. We can see that it starts this way.

```
• (venv) ignagonmell@DESKTOP-  
[0 0] [0 0]  
[1. 0.] [0. 1.]  
[1. 1.] [0. 2.]  
[1. 2.] [0. 3.]  
[1. 3.] [0. 4.]  
[1. 4.] [0. 5.]  
[1. 5.] [0. 6.]  
[1. 6.] [0. 7.]  
[1. 7.] [0. 8.]  
[1. 8.] [0. 9.]  
[1. 9.] [ 0. 10.]
```

With this we can conclude that the equilibrium tends to be a dominant strategy of betrayal, as similar as exercise 1. However, in this exercise the equilibrium is computed several times starting with a $[1, 0]$, $[0, 1]$ strategy and progressively adapting to the final result.